

DMSL Problem Statements – Answers

Source File: [filecite2turn0file02](#)

1. An institute maintains details of all students, including name, roll number, address, exam_date, marks of 3 subjects (PCM), date of admission, and scholarship details. Create a MongoDB queries collection. Identify the proper datatypes of each field. Justify and write following queries.

Collection Creation

```
db.students.createCollection("students")
```

Sample Document

```
db.students.insertOne({
  name: "Amit",
  roll_no: 101,
  address: {
    city: "Pune",
    state: "Maharashtra"
  },
  exam_date: ISODate("2026-02-18"),
  marks: {
    physics: 85,
    chemistry: 90,
    maths: 95
  },
  admission_date: ISODate("2023-06-15"),
  scholarship: "National Merit"
})
```

Datatypes

- name → String
- roll_no → Integer
- address → Object
- exam_date → Date
- marks → Object
- admission_date → Date
- scholarship → String

Queries

1. Count students admitted in 2020

```
db.students.countDocuments({
  admission_date: {
    $gte: ISODate("2020-01-01"),
    $lt: ISODate("2021-01-01")
  }
})
```

2. Students with National Merit scholarship

```
db.students.find({ scholarship: "National Merit" }, { name: 1, _id: 0 })
```

3. Students from Maharashtra

```
db.students.find({ "address.state": "Maharashtra" })
```

4. Increase physics marks by 6

```
db.students.updateMany(
  { exam_date: ISODate("2026-02-18") },
  { $inc: { "marks.physics": 6 } }
)
```

5. Delete students with maths < 60

```
db.students.deleteMany({ "marks.maths": { $lt: 60 } })
```

6. Average marks > 85

```
db.students.aggregate([
  {
    $project: {
      name: 1,
      average: {
        $avg: ["$marks.physics", "$marks.chemistry", "$marks.maths"]
      }
    }
  },
  { $match: { average: { $gt: 85 } } }
])
```

7. Topper

```
db.students.aggregate([
  {
    $project: {
      name: 1,
      total: {
        $add: ["$marks.physics", "$marks.chemistry", "$marks.maths"]
      }
    }
  }
])
```

```

    },
    { $sort: { total: -1 } },
    { $limit: 1 }
  ])

```

8. Count city-wise

```

db.students.aggregate([
  { $group: { _id: "$address.city", total: { $sum: 1 } } }
])

```

9. Highest marks in each subject

```

db.students.aggregate([
  {
    $group: {
      _id: null,
      maxPhysics: { $max: "$marks.physics" },
      maxChemistry: { $max: "$marks.chemistry" },
      maxMaths: { $max: "$marks.maths" }
    }
  }
])

```

10. Lowest maths marks

```

db.students.aggregate([
  { $group: { _id: null, minMaths: { $min: "$marks.maths" } } }
])

```

11. Students admitted after 2022

```

db.students.find({ admission_date: { $gt: ISODate("2022-12-31") } })

```

12. Physics marks between 80 and 90

```

db.students.find({ "marks.physics": { $gte: 80, $lte: 90 } })

```

13. Name starts with A

```

db.students.find({ name: /^A/ })

```

14. City not Pune or Mumbai

```

db.students.find({ "address.city": { $nin: ["Pune", "Mumbai"] } })

```

15. Add new subject marks

```

db.students.updateOne(
  { roll_no: 101 },
  { $set: { "marks.biology": 88 } }
)

```

16. Delete students admitted after 1 Aug 2025

```
db.students.deleteMany({
  admission_date: { $gt: ISODate("2025-08-01") }
})
```

2. Consider schema to maintain movie information including movie_name, rating_star, movie_cast, release_date, budget, collection_amount, awards. Create a MongoDB queries collection. Identify the appropriate data types for each field. Justify and write following queries.

Sample Document

```
db.movies.insertOne({
  movie_name: "Avengers",
  rating_star: 4.8,
  movie_cast: ["Robert", "Chris"],
  release_date: ISODate("2020-02-10"),
  budget: 500000,
  collection_amount: 800000,
  awards: ["International", "National"]
})
```

Queries

1. Movies released in 2020

```
db.movies.countDocuments({
  release_date: {
    $gte: ISODate("2020-01-01"),
    $lt: ISODate("2021-01-01")
  }
})
```

2. Movies with both awards

```
db.movies.find({
  awards: { $all: ["International", "National"] }
}, { movie_name: 1, _id: 0 })
```

3. Movies rating > 4.2

```
db.movies.find({ rating_star: { $gt: 4.2 } })
```

4. Collection less than budget

```
db.movies.find({
  $expr: { $lt: ["$collection_amount", "$budget"] }
}, { movie_name: 1 })
```

5. Delete movies rating < 2

```
db.movies.deleteMany({ rating_star: { $lt: 2 } })
```

6. Give international award to movies rating > 4.8

```
db.movies.updateMany(
  { rating_star: { $gt: 4.8 } },
  { $addToSet: { awards: "International" } }
)
```

3. Consider following structure for MongoDB collections and write a query for following requirements in MongoDB. Identify the appropriate data types for each field. Book (Bookname, Authors, Publisher, Edition, Year, book_rating, no_of_copies)

Create Collection

```
db.createCollection("books")
```

Sample Insert

```
db.books.insertMany([
  {
    Bookname: "Database System",
    Authors: "Balkrishna",
    Publisher: "Tech",
    Edition: 2,
    Year: 2018,
    book_rating: 4.7,
    no_of_copies: 50
  },
  {
    Bookname: "Fundamental C",
    Authors: "Balkrishna",
    Publisher: "ABC",
    Edition: 1,
    Year: 2026,
    book_rating: 4.8,
  }
])
```

```
    no_of_copies: 30
  }
])
```

ii) Books of Database

```
db.books.find({ Bookname: /Database/i })
```

iii) Author Balkrishna and name Fundamental C

```
db.books.find({
  Authors: "Balkrishna",
  Bookname: "Fundamental C"
})
```

iv) Total books published in 2018

```
db.books.countDocuments({ Year: 2018 })
```

v) Rating greater than 4.5

```
db.books.countDocuments({ book_rating: { $gt: 4.5 } })
```

vi) Delete books before 1999

```
db.books.deleteMany({ Year: { $lt: 1999 } })
```

vii) Decrease copies by 5

```
db.books.updateMany(
  { Year: 2026 },
  { $inc: { no_of_copies: -5 } }
)
```

An institute maintains details of all students, including name, roll number, address, exam_date, marks of 3 subjects (PCM), date of admission, and scholarship details. Create a MongoDB queries collection. Identify the appropriate data types for each field and justify. Create various index on it and demonstrate its impact while retrieving the records.

Create Indexes

```
db.students.createIndex({ name: 1 })
db.students.createIndex({ roll_no: 1 }, { unique: true })
db.students.createIndex({ "address.city": 1 })
db.students.createIndex({ admission_date: -1 })
```

Explain Query

```
db.students.find({ name: "Amit" }).explain("executionStats")
```

Consider schema to maintain movie information including movie_name, rating_star, movie_cast, release_date, budget, colletion_amount, awards. Create a MongoDB queries collection. Identify the proper datatypes of each field and Justify. Create various index on it and demonstrate its impact while retrieving the records.

```
db.movies.createIndex({ movie_name: 1 })
db.movies.createIndex({ rating_star: -1 })
db.movies.createIndex({ release_date: 1 })
```

Consider following structure for MongoDB collections and write a query for following requirements in MongoDB. Book (Bookname, subjectName, Author, Publisher, Edition, Year, price)

1. Index on Author

```
db.books.createIndex({ Author: 1 })
```

2. Compound Index

```
db.books.createIndex({ Author: 1, Bookname: 1 })
```

3. Recently published book

```
db.books.find().sort({ Year: -1 }).limit(1)
```

4. Price > 500

```
db.books.find({ price: { $gt: 500 } })
```

5. Update price of Fundamental C

```
db.books.updateOne(
  { Bookname: "Fundamental C" },
  { $set: { price: 700 } }
)
```

6. Delete books price < 500

```
db.books.deleteMany({ price: { $lt: 500 } })
```

7. Average price

```
db.books.aggregate([
  { $group: { _id: null, avgPrice: { $avg: "$price" } } }
])
```

8. Count books by subject

```
db.books.aggregate([
  { $group: { _id: "$subjectName", count: { $sum: 1 } } }
])
```

9. Price range 500–700

```
db.books.find({ price: { $gte: 500, $lte: 700 } })
```


10. Sort newest first

```
db.books.find().sort({ Year: -1 })
```

7. Write a program to demonstrate the JAVA connectivity with MySQL database using JDBC Connectivity. Consider the relation storing Movie information including movie_id, movie_name, director_name, star_rating, release_date. Perform following operation on it

1. Insert record
2. Display all record
3. Update star_rating of “XYZ” movie
4. Display all information of movie directed by “ABC”

Create Table

```
CREATE TABLE movie(  
  movie_id INT PRIMARY KEY,  
  movie_name VARCHAR(50),  
  director_name VARCHAR(50),  
  star_rating FLOAT,  
  release_date DATE  
);
```

Java Program

```
import java.sql.*;  
  
public class MovieJDBC {  
    public static void main(String[] args) throws Exception {  
        Connection con = DriverManager.getConnection(  
            "jdbc:mysql://localhost:3306/test",  
            "root",  
            "root"  
        );  
  
        Statement st = con.createStatement();
```

```

        st.executeUpdate("insert into movie values(1,'XYZ','ABC',4.5,'2020-01-10')");

        ResultSet rs = st.executeQuery("select * from movie");
        while(rs.next()) {
            System.out.println(rs.getString(2));
        }

        st.executeUpdate("update movie set star_rating=5 where movie_name='XYZ'");

        rs = st.executeQuery("select * from movie where director_name='ABC'");

        con.close();
    }
}

```

8. Write a program to demonstrate the JAVA connectivity with MySQL database using JDBC Connectivity. Consider the relation storing Movie information, including movie_id, movie_name, director_name, star_rating, and release_date. Perform the following operation on it

1. Insert record
2. Delete any specific record
3. Display all record
4. Display all information of movie released in the year 2020

```

INSERT INTO movie VALUES(2,'Avatar','James',4.9,'2020-05-01');
DELETE FROM movie WHERE movie_id=2;
SELECT * FROM movie;
SELECT * FROM movie WHERE YEAR(release_date)=2020;

```

9. Write a program to demonstrate the JAVA connectivity with MySQL database using JDBC Connectivity. Consider the relation storing Book information (Bookname, Author_name, Publisher, Edition, Year, book_rating, price) Perform following operation on it

1. Insert record

2. Delete any specific record

3. Display all record

4. Display information of all the books written by “ Korth”

```
INSERT INTO books VALUES('DBMS','Korth','McGraw',5,2020,4.8,500);  
DELETE FROM books WHERE Bookname='DBMS';  
SELECT * FROM books;  
SELECT * FROM books WHERE Author_name='Korth';
```

10. Write a program to demonstrate the JAVA connectivity with MySQL database using JDBC Connectivity. Consider the relation storing Book information (Bookname, Author_name, Publisher, Edition, Year, book_rating, price) Perform following operation on it

1. Insert record

2. Increase price of books by 5% if published after 2022

3. Display all record

4. Display information of all the books having price > 350

```
INSERT INTO books VALUES('Java','James','Tech',2,2023,4.5,400);
UPDATE books SET price = price + (price*0.05) WHERE Year > 2022;
SELECT * FROM books;
SELECT * FROM books WHERE price > 350;
```

11. Write a program to demonstrate the JAVA connectivity with MySQL database using JDBC Connectivity. Consider the relation storing Movie information including movie_id, movie_name, director_name, star_rating, release_date. Perform following operation on it

1. Insert record

2. Extend release_date by one week for given specific movie.

3. Display all record

4. Display all information of movie released in the year 2020

```
UPDATE movie
SET release_date = DATE_ADD(release_date, INTERVAL 7 DAY)
WHERE movie_name='XYZ';
```

12. An institute maintains details of all teachers, including name, qualifications, department details, experience, salary structure, date of joining, appointment_nature and area of expertise. Design and implement MongoDB queries to perform CRUD operations on the teachers collection for various administrative tasks. Create the above collection, insert suitable documents and design updation and retrieval queries requiring comparison and logical operators, save() method, etc.

Sample Insert

```
db.teachers.insertOne({
  name: "Raman",
  qualification: "PhD",
  department: "Computer",
  experience: 15,
  salary: {
    basic: 50000,
    hra: 10000
  },
  date_of_joining: ISODate("2010-06-01"),
  appointment_nature: "Permanent",
  expertise: ["DBMS"]
})
```

Queries

1. Add document if not exists

```
db.teachers.updateOne(
  { name: "Raman" },
  { $setOnInsert: { qualification: "PhD" } },
  { upsert: true }
)
```

2. Increment basic pay by 10%

```
db.teachers.updateMany({}, [
  {
    $set: {
      "salary.basic": {
        $multiply: ["$salary.basic", 1.10]
      }
    }
  }
])
```

```
    }  
  }  
}  
])
```

3. Add expertise

```
db.teachers.updateOne(  
  { name: "Raman" },  
  { $addToSet: { expertise: "AI" } }  
)
```

4. Descending experience

```
db.teachers.find().sort({ experience: -1 })
```

5. Change department

```
db.teachers.updateOne(  
  { name: "Anita" },  
  { $set: { department: "Information Technology" } }  
)
```

6. Delete teachers with experience 2–3

```
db.teachers.deleteMany({  
  experience: { $gte: 2, $lte: 3 }  
})
```

7. Experience >20 or PhD

```
db.teachers.find({  
  $or: [  
    { experience: { $gt: 20 } },  
    { qualification: "PhD" }  
  ]  
})
```

13. The University wants to analyze course enrollments, faculty performance and student achievements using MongoDB aggregation pipelines. The university maintains a “courses” collection, which includes: i. Course details (title, department, credit hours, instructor) ii. Student enrollments (student names, scores, pass/fail status) iii. Faculty details (experience, designation, department) Design and implement flowing aggregation queries to generate various reports using multiple aggregation stages with the given collection in MongoDB.

1. Count courses by instructor

```
db.courses.aggregate([
  { $group: { _id: "$instructor", totalCourses: { $sum: 1 } } }
])
```

2. Average marks in every course

```
db.courses.aggregate([
  {
    $group: {
      _id: "$title",
      avgMarks: { $avg: "$student.score" }
    }
  }
])
```

3. Faculty descending experience

```
db.courses.aggregate([
  { $sort: { "faculty.experience": -1 } }
])
```

4. Students with highest score

```
db.courses.aggregate([
  { $sort: { "student.score": -1 } },
  { $limit: 1 }
])
```

5. Pass percentage

```
db.courses.aggregate([
  {
    $group: {
      _id: "$title",
      total: { $sum: 1 },
      passCount: {
        $sum: {
          $cond: [{ $eq: ["$student.status", "Pass"] }, 1, 0]
        }
      }
    }
  },
  {
    $project: {
      passPercentage: {
        $multiply: [{ $divide: ["$passCount", "$total"] }, 100]
      }
    }
  }
])
```

1. Write a Stored Procedure namely proc_Grade for the categorization of customer. If purchase by customer in year is ≤ 20000 and ≥ 10000 then customer will be placed in platinum category. If purchase by customer is between 9999 and 5000 category is gold, if purchase between 4999 and 2000 category is silver. Write a PL/SQL block for using procedure created with above requirement. Customer(Cust_id,name, total_purchase) Category(Cust_id,Name,Class)

DELIMITER //

```
CREATE PROCEDURE proc_Grade()
BEGIN
  INSERT INTO Category
  SELECT Cust_id, name,
  CASE
```



```

        WHEN total_purchase BETWEEN 10000 AND 20000 THEN 'Platinum'
        WHEN total_purchase BETWEEN 5000 AND 9999 THEN 'Gold'
        WHEN total_purchase BETWEEN 2000 AND 4999 THEN 'Silver'
        ELSE 'Normal'
    END
    FROM Customer;
END //

```

DELIMITER ;

Call Procedure

```
CALL proc_Grade();
```

2. Write a database trigger on Student table. The System should keep track of the records that are being updated or deleted. The old value of updated or deleted records should be added in Alumni table.

Student(Rollno,Name,DateofAdmission,branch,percent,S
tatus) Departments (dept_id , dept_name)

Professors(prof_id,prof_fname,prof_lname,dept_id,design
ation,salary,do j,email,phone,city)

works(prof_id,dept_id,duration) Shift
(prof_id,shift,working_hours)

1. Insert atleast 10 records in professors table and insert other tables accordingly.

2. Display all professors details with city pune and mumbai and professor first name starting with 'a' or 'd'.

3. List the number of different cities of professors.

4. Give 5% increase in salary of the professors with date of joining 1-1-2015.
5. Delete professor details living in pune.
6. Find the names of professors belonging to pune or mumbai(use of UNION)
7. Find the professor having maximum salary and names of professors having salary between 10,000 and 20,000.
8. Display all professors name with salary and date of joining with decreasing order of salary.
9. Display professors name, date of joining and dept_id with salary 30000, 40000 and 50000
10. Find the professor details and department details.
11. Find the prof_id, prof_name and shift.
12. List all the department details and the corresponding names of professors in the same department.
13. List all the professors and the corresponding names of department.
14. Display professor name, dept_name, shift, salary where prof_id = 101
15. list the total number of professor in each department.

16. List the prof_id associated department and the dept_name having name 'computer'
17. Find the names of all departments where the professors joined in year 2015 (or date of joining is 1-1-2015)
18. Create view showing the professor and shift details.

```
CREATE TABLE Alumni(  
  Rollno INT,  
  Name VARCHAR(30),  
  DateofAdmission DATE,  
  branch VARCHAR(20),  
  percent FLOAT,  
  Status VARCHAR(20)  
);  
  
DELIMITER //  
CREATE TRIGGER trg_student_delete  
BEFORE DELETE ON Student  
FOR EACH ROW  
BEGIN  
  INSERT INTO Alumni  
  VALUES(OLD.Rollno, OLD.Name, OLD.DateofAdmission,  
  OLD.branch, OLD.percent, OLD.Status);  
END //  
DELIMITER ;
```

Professors SQL Queries

1. Insert sample records

```
INSERT INTO Professors VALUES  
(101, 'Amit', 'Patil', 1, 'Professor', 50000, '2015-01-  
01', 'amit@gmail.com', '9876543210', 'Pune');
```

2. Pune/Mumbai and name starts A or D

```
SELECT * FROM Professors  
WHERE city IN ('Pune', 'Mumbai')  
AND (prof_fname LIKE 'A%' OR prof_fname LIKE 'D%');
```

3. Count different cities

```
SELECT COUNT(DISTINCT city) FROM Professors;
```

4. Increase salary by 5%

```
UPDATE Professors  
SET salary = salary + salary*0.05  
WHERE doj='2015-01-01';
```

5. Delete professors from Pune

```
DELETE FROM Professors WHERE city='Pune';
```

6. Professors from Pune or Mumbai

```
SELECT prof_fname FROM Professors WHERE city='Pune'  
UNION  
SELECT prof_fname FROM Professors WHERE city='Mumbai';
```

7. Maximum salary and salary range

```
SELECT * FROM Professors  
WHERE salary=(SELECT MAX(salary) FROM Professors);
```

```
SELECT * FROM Professors  
WHERE salary BETWEEN 10000 AND 20000;
```

8. Salary descending

```
SELECT prof_fname,salary,doj  
FROM Professors  
ORDER BY salary DESC;
```

9. Salary 30000,40000,50000

```
SELECT prof_fname,doj,dept_id  
FROM Professors  
WHERE salary IN (30000,40000,50000);
```

10. Professor and department details

```
SELECT *  
FROM Professors p  
JOIN Departments d  
ON p.dept_id=d.dept_id;
```

11. prof_id, name and shift

```
SELECT p.prof_id,p.prof_fname,s.shift  
FROM Professors p  
JOIN Shift s  
ON p.prof_id=s.prof_id;
```

12. Department details and professor names

```
SELECT d.*,p.prof_fname
FROM Departments d
JOIN Professors p
ON d.dept_id=p.dept_id;
```

13. All professors and departments

```
SELECT p.prof_fname,d.dept_name
FROM Professors p
LEFT JOIN Departments d
ON p.dept_id=d.dept_id;
```

14. Professor name, dept_name, shift, salary

```
SELECT p.prof_fname,d.dept_name,s.shift,p.salary
FROM Professors p
JOIN Departments d ON p.dept_id=d.dept_id
JOIN Shift s ON p.prof_id=s.prof_id
WHERE p.prof_id=101;
```

15. Total professors in each department

```
SELECT dept_id,COUNT(*)
FROM Professors
GROUP BY dept_id;
```

16. Department having name computer

```
SELECT p.prof_id,d.dept_name
FROM Professors p
JOIN Departments d
ON p.dept_id=d.dept_id
WHERE d.dept_name='computer';
```

17. Departments where professors joined in 2015

```
SELECT DISTINCT d.dept_name
FROM Departments d
JOIN Professors p
ON d.dept_id=p.dept_id
WHERE YEAR(p.doj)=2015;
```

18. View for professor and shift

```
CREATE VIEW professor_shift AS
SELECT p.prof_fname,s.shift
FROM Professors p
JOIN Shift s
ON p.prof_id=s.prof_id;
```

1. Department(dept_id,dept_name, building, loc)
student(stud_id,stud_name,email,phone,dob,dept_id);
Instructor(instructor_id, instructor_name, dept_id, salary)

1. List students and their department names

```
SELECT s.stud_name, d.dept_name
FROM student s
JOIN Department d
ON s.dept_id = d.dept_id;
```

2. Display the total number of students belonging to each department

```
SELECT d.dept_name, COUNT(s.stud_id) AS total_students
FROM Department d
LEFT JOIN student s
ON d.dept_id = s.dept_id
GROUP BY d.dept_name;
```

3. List the department names that do not have any students

```
SELECT d.dept_name
FROM Department d
LEFT JOIN student s
ON d.dept_id = s.dept_id
WHERE s.stud_id IS NULL;
```

4. Find the Instructor(s) with the highest salary

```
SELECT *
FROM Instructor
WHERE salary = (SELECT MAX(salary) FROM Instructor);
```

5. Create a view to retrieve the email and phone number of all departments present at location “Pune”

```
CREATE VIEW pune_department_contacts AS
SELECT s.email, s.phone
FROM student s
JOIN Department d
ON s.dept_id = d.dept_id
WHERE d.loc = 'Pune';
```

6. Increase the salary of all instructors from the “IT” department by 20%

```
UPDATE Instructor
SET salary = salary + (salary * 0.20)
WHERE dept_id IN (
    SELECT dept_id
    FROM Department
    WHERE dept_name = 'IT'
```

);

2. student(stud_id,stud_name,email,phone,dob);
Course(course_id, course_name, credits,instructor_id)
Enrollment(enrollement_id, student_id, course_id,
date_of_enrollement) Instructor(instructor_id,
instructor_name, salary) 1. List all students who have enrolled for
any course

```
SELECT DISTINCT s.*  
FROM student s  
JOIN Enrollment e  
ON s.stud_id = e.student_id;
```

2. Find students enrolled in the course "DBMS"

```
SELECT s.stud_name  
FROM student s  
JOIN Enrollment e  
ON s.stud_id = e.student_id  
JOIN Course c  
ON e.course_id = c.course_id  
WHERE c.course_name = 'DBMS';
```

3. Find students enrolled in more than one course

```
SELECT s.stud_name, COUNT(e.course_id) AS total_courses  
FROM student s  
JOIN Enrollment e  
ON s.stud_id = e.student_id  
GROUP BY s.stud_name  
HAVING COUNT(e.course_id) > 1;
```

4. Find the list of students enrolled in the same course as "John"

```
SELECT DISTINCT s2.stud_name  
FROM student s1  
JOIN Enrollment e1  
ON s1.stud_id = e1.student_id  
JOIN Enrollment e2  
ON e1.course_id = e2.course_id  
JOIN student s2  
ON e2.student_id = s2.stud_id  
WHERE s1.stud_name = 'John'  
AND s2.stud_name <> 'John';
```

5. Find the Instructor(s) with the lowest salary

```
SELECT *  
FROM Instructor  
WHERE salary = (SELECT MIN(salary) FROM Instructor);
```

6. Create a view of instructors and their courses

```
CREATE VIEW instructor_course_view AS
SELECT i.instructor_name, c.course_name
FROM Instructor i
JOIN Course c
ON i.instructor_id = c.instructor_id;
```

3. student(stud_id,stud_name,email,phone,dob);
Course(course_id, course_name, credits,instructor_id)
Enrollment(enrollement_id, student_id, course_id,
date_of_enrollement) Instructor(instructor_id,
instructor_name, salary)

1. List the course for which there is no enrollment of students

```
SELECT c.course_name
FROM Course c
LEFT JOIN Enrollment e
ON c.course_id = e.course_id
WHERE e.course_id IS NULL;
```

2. Find students enrolled in the course "DBMS"

```
SELECT s.stud_name
FROM student s
JOIN Enrollment e
ON s.stud_id = e.student_id
JOIN Course c
ON e.course_id = c.course_id
WHERE c.course_name = 'DBMS';
```

3. Find instructors who teach more than one course

```
SELECT i.instructor_name, COUNT(c.course_id) AS total_courses
FROM Instructor i
JOIN Course c
ON i.instructor_id = c.instructor_id
GROUP BY i.instructor_name
HAVING COUNT(c.course_id) > 1;
```

4. Find the list of students enrolled in the same course as “James”

```
SELECT DISTINCT s2.stud_name
FROM student s1
JOIN Enrollment e1
ON s1.stud_id = e1.student_id
JOIN Enrollment e2
ON e1.course_id = e2.course_id
JOIN student s2
ON e2.student_id = s2.stud_id
```



```
WHERE s1.stud_name = 'James'
AND s2.stud_name <> 'James';
```

5. Find the Instructor(s) with the lowest salary

```
SELECT *
FROM Instructor
WHERE salary = (SELECT MIN(salary) FROM Instructor);
```

6. Create a view of students and their courses

```
CREATE VIEW student_course_view AS
SELECT s.stud_name, c.course_name
FROM student s
JOIN Enrollment e
ON s.stud_id = e.student_id
JOIN Course c
ON e.course_id = c.course_id;
```

4. Department(dept_id,dept_name, building, loc) Course(course_id, course_name, dept_name, credits,instructor_id) Instructor(instructor_id, instructor_name, dept_id, salary)

1. Display instructors' details and their department names

```
SELECT i.*, d.dept_name
FROM Instructor i
JOIN Department d
ON i.dept_id = d.dept_id;
```

2. Display the total number of instructors belonging to each department

```
SELECT d.dept_name, COUNT(i.instructor_id) AS total_instructors
FROM Department d
LEFT JOIN Instructor i
ON d.dept_id = i.dept_id
GROUP BY d.dept_name;
```

3. Create a view to list the department names that do not have any instructors

```
CREATE VIEW dept_without_instructors AS
SELECT d.dept_name
FROM Department d
LEFT JOIN Instructor i
ON d.dept_id = i.dept_id
WHERE i.instructor_id IS NULL;
```

4. Display the courses with credits greater than 3 and instructors' details handling them

```
SELECT c.course_name, c.credits, i.*
FROM Course c
JOIN Instructor i
ON c.instructor_id = i.instructor_id
WHERE c.credits > 3;
```

5. Find instructors who teach more than one course

```
SELECT i.instructor_name, COUNT(c.course_id) AS total_courses
FROM Instructor i
JOIN Course c
ON i.instructor_id = c.instructor_id
GROUP BY i.instructor_name
HAVING COUNT(c.course_id) > 1;
```

6. Find Instructor(s) having salary greater than average salary

```
SELECT *
FROM Instructor
WHERE salary > (SELECT AVG(salary) FROM Instructor);
```

5. Department(dept_id,dept_name, building, loc)
Course(course_id, course_name, dept_name,
credits,instructor_id) Enrollment(enrollement_id,
course_id, date_of_enrollement) Instructor(instructor_id,
instructor_name, dept_id, salary)

1. Display the courses with credits greater than 3 and instructors handling them

```
SELECT c.course_name, c.credits, i.instructor_name
FROM Course c
JOIN Instructor i
ON c.instructor_id = i.instructor_id
WHERE c.credits > 3;
```

2. List instructors with department names in descending order of salary

```
SELECT i.instructor_name, d.dept_name, i.salary
FROM Instructor i
JOIN Department d
ON i.dept_id = d.dept_id
ORDER BY i.salary DESC;
```

3. Find instructors who teach more than one course

```
SELECT i.instructor_name, COUNT(c.course_id)
FROM Instructor i
JOIN Course c
ON i.instructor_id = c.instructor_id
GROUP BY i.instructor_name
HAVING COUNT(c.course_id) > 1;
```

4. Find Instructor(s) with salary less than average salary

```
SELECT *
FROM Instructor
WHERE salary < (SELECT AVG(salary) FROM Instructor);
```

5. Create a view of instructors and their courses enrolled after '2020-01-01'

```
CREATE VIEW instructor_course_after2020 AS
SELECT i.instructor_name, c.course_name, e.date_of_enrollement
FROM Instructor i
JOIN Course c
ON i.instructor_id = c.instructor_id
JOIN Enrollment e
ON c.course_id = e.course_id
WHERE e.date_of_enrollement > '2020-01-01';
```

6. Decrease salary of CE department instructors by 10%

```
UPDATE Instructor
SET salary = salary - (salary * 0.10)
WHERE dept_id IN (
    SELECT dept_id
    FROM Department
    WHERE dept_name = 'CE'
);
```

6. Customer(cust_id, cust_name, cust_phoneNo, cust_emailID, cust_city, cust_created_date)
Product(product_id, product_name, product_category, product_price)
Orders(order_id, customer_id, product_id, order_date, quantity, amount)

1. Find products with Electronics, Stationary or Cloth category

```
SELECT *
FROM Product
WHERE product_category IN ('Electronics', 'Stationary', 'Cloth');
```

2. Find customers who never placed an order

```
SELECT *
FROM Customer
WHERE cust_id NOT IN (
    SELECT customer_id
    FROM Orders
);
```

3. Find names of 5 most expensive products

```
SELECT product_name, product_price
FROM Product
ORDER BY product_price DESC
FETCH FIRST 5 ROWS ONLY;
```

4. Display customer names who placed orders worth more than 10000

```
SELECT DISTINCT c.cust_name
FROM Customer c
JOIN Orders o
ON c.cust_id = o.customer_id
WHERE o.amount > 10000;
```

5. Increase Electronics product price by 10%

```
UPDATE Product
SET product_price = product_price + (product_price * 0.10)
WHERE product_category = 'Electronics';
```

6. Delete orders placed before 2020

```
DELETE FROM Orders
WHERE order_date < '2020-01-01';
```

7. Customer(cust_id, cust_name, cust_phoneNo,
cust_emailID, cust_city, cust_created_date)
Product(product_id, product_name, product_category,
product_price) Orders(order_id, customer_id, product_id,
order_date, quantity, amount)

1. List different categories

```
SELECT DISTINCT product_category
FROM Product;
```

2. Find Electronics products priced between 1000 and 5000

```
SELECT *
FROM Product
WHERE product_category = 'Electronics'
```

```
AND product_price BETWEEN 1000 AND 5000;
```

3. Find order details where product name starts with E or C

```
SELECT o.*
FROM Orders o
JOIN Product p
ON o.product_id = p.product_id
WHERE p.product_name LIKE 'E%'
OR p.product_name LIKE 'C%';
```

4. Calculate total amount spent by each customer

```
SELECT c.cust_name, SUM(o.amount) AS total_spent
FROM Customer c
JOIN Orders o
ON c.cust_id = o.customer_id
GROUP BY c.cust_name;
```

5. Display products with price category

```
SELECT product_name,
CASE
    WHEN product_price < 200 THEN 'Cheap'
    WHEN product_price BETWEEN 200 AND 1000 THEN 'Moderate'
    ELSE 'Expensive'
END AS price_category
FROM Product;
```

6. Delete customers who never placed any order

```
DELETE FROM Customer
WHERE cust_id NOT IN (
    SELECT customer_id
    FROM Orders
);
```

7. Delete all customers

```
DELETE FROM Customer;
```

8. Customer(cust_id, cust_name, cust_phoneNo,
cust_emailID, cust_city, cust_created_date)
Product(product_id, product_name, product_category,
product_price) Orders(order_id, customer_id, product_id,
order_date, quantity, amount)

1. Find products with Electronics, Stationary or Cloth category

```
SELECT *
FROM Product
```

```
WHERE product_category IN ('Electronics', 'Stationary', 'Cloth');
```

2. Find customers who never placed an order

```
SELECT *  
FROM Customer  
WHERE cust_id NOT IN (  
    SELECT customer_id  
    FROM Orders  
);
```

3. Find names of 5 most expensive products

```
SELECT product_name, product_price  
FROM Product  
ORDER BY product_price DESC  
FETCH FIRST 5 ROWS ONLY;
```

4. Create a view to display customers with orders more than 10000

```
CREATE VIEW high_value_customers AS  
SELECT DISTINCT c.cust_name  
FROM Customer c  
JOIN Orders o  
ON c.cust_id = o.customer_id  
WHERE o.amount > 10000;
```

5. Display products with price category

```
SELECT product_name,  
CASE  
    WHEN product_price < 200 THEN 'Cheap'  
    WHEN product_price BETWEEN 200 AND 1000 THEN 'Moderate'  
    ELSE 'Expensive'  
END AS price_category  
FROM Product;
```

6. Increase Electronics product price by 10%

```
UPDATE Product  
SET product_price = product_price + (product_price * 0.10)  
WHERE product_category = 'Electronics';
```

7. Delete orders placed after '2020-01-01'

```
DELETE FROM Orders  
WHERE order_date > '2020-01-01';
```

9. A Bank wants to automate its customer credit scoring system based on their financial standing. Let the credit score be computed on a scale of 0 to 5 by a PLSQL function based on the account balance and loan amount of any customer. (hint: if loan_amount is 50% of account balance then, credit_score: 3) Write a stored procedure that uses a cursor and calls the function with each customer name for computing credit scores of all bank customers

Function

```
CREATE OR REPLACE FUNCTION get_credit_score(
    balance NUMBER,
    loan_amount NUMBER
)
RETURN NUMBER
IS
    score NUMBER;
BEGIN
    IF loan_amount <= balance * 0.25 THEN
        score := 5;
    ELSIF loan_amount <= balance * 0.50 THEN
        score := 3;
    ELSE
        score := 1;
    END IF;

    RETURN score;
END;
/
```

Procedure using Cursor

```
CREATE OR REPLACE PROCEDURE calculate_credit_scores
IS
    CURSOR c1 IS
        SELECT c.Cust_name, c.Balance, l.Amount, b.Loan_no
        FROM Customer c
        JOIN Borrower b
        ON c.Cust_name = b.Cust_name
        JOIN Loan l
        ON b.Loan_no = l.Loan_no;

    v_score NUMBER;
BEGIN
    FOR rec IN c1 LOOP
```

```

        v_score := get_credit_score(rec.Balance, rec.Amount);

        UPDATE Borrower
        SET CreditScore = v_score
        WHERE Loan_no = rec.Loan_no
        AND Cust_name = rec.Cust_name;
    END LOOP;
END;
/

```

10. A Bank wants to automate its customer credit scoring system based on their financial standing. Write a stored procedure that uses a cursor to display each customer's name, balance and loan amount.

```

CREATE OR REPLACE PROCEDURE display_customer_loan
IS
    CURSOR c1 IS
        SELECT c.Cust_name, c.Balance, l.Amount
        FROM Customer c
        JOIN Borrower b
        ON c.Cust_name = b.Cust_name
        JOIN Loan l
        ON b.Loan_no = l.Loan_no;
BEGIN
    FOR rec IN c1 LOOP
        DBMS_OUTPUT.PUT_LINE(
            rec.Cust_name || ' ' ||
            rec.Balance || ' ' ||
            rec.Amount
        );
    END LOOP;
END;
/

```

11. A Bank wants to automate its customer credit scoring system based on their financial standing. Let the credit score be computed on a scale of 0 to 5 by a PLSQL function based on the account balance and loan amount of any customer. (hint: if loan_amount is 50% of account balance then, credit_score: 3).

```

CREATE OR REPLACE FUNCTION get_credit_score(
    balance NUMBER,
    loan_amount NUMBER

```



```

)
RETURN NUMBER
IS
BEGIN
    IF loan_amount <= balance * 0.25 THEN
        RETURN 5;
    ELSIF loan_amount <= balance * 0.50 THEN
        RETURN 3;
    ELSE
        RETURN 1;
    END IF;
END;
/

```

12. A Company wants to track employee salary changes, maintain companywide statistics, and log employment history efficiently using row-level triggers. Implement BEFORE and AFTER triggers on EMPLOYEE and EMP_LOG tables using INSERT and DELETE operations.

BEFORE INSERT Trigger

```

CREATE OR REPLACE TRIGGER trg_before_insert
BEFORE INSERT ON EMPLOYEE
FOR EACH ROW
BEGIN
    :NEW.HRA := :NEW.Salary * 0.10;
END;
/

```

AFTER INSERT Trigger

```

CREATE OR REPLACE TRIGGER trg_after_insert
AFTER INSERT ON EMPLOYEE
FOR EACH ROW
BEGIN
    INSERT INTO EMP_LOG
    VALUES (
        :NEW.Emp_Id,
        :NEW.Salary,
        :NEW.Hire_Date,
        NULL,
        'ACTIVE'
    );
END;
/

```

AFTER DELETE Trigger

```

CREATE OR REPLACE TRIGGER trg_after_delete

```

```

AFTER DELETE ON EMPLOYEE
FOR EACH ROW
BEGIN
    INSERT INTO EMP_LOG
    VALUES (
        :OLD.Emp_Id,
        :OLD.Salary,
        :OLD.Hire_Date,
        SYSDATE,
        'LEFT'
    );
END;
/

```

13. A Company wants to track employee salary changes, maintain companywide statistics, and log employment history efficiently using row-level triggers. Implement BEFORE and AFTER triggers on EMPLOYEE and EMP_LOG tables using INSERT and DELETE operations.

AFTER INSERT Trigger

```

CREATE OR REPLACE TRIGGER trg_company_insert
AFTER INSERT ON EMPLOYEE
FOR EACH ROW
BEGIN
    UPDATE COMPANY_INFO
    SET Emp_Count = Emp_Count + 1,
        Total_Salary_Expenses =
        Total_Salary_Expenses + :NEW.Salary;
END;
/

```

AFTER DELETE Trigger

```

CREATE OR REPLACE TRIGGER trg_company_delete
AFTER DELETE ON EMPLOYEE
FOR EACH ROW
BEGIN
    UPDATE COMPANY_INFO
    SET Emp_Count = Emp_Count - 1,
        Total_Salary_Expenses =
        Total_Salary_Expenses - :OLD.Salary;
END;
/

```

14. A Company wants to track employee salary changes, maintain companywide statistics, and log employment history efficiently using row-level triggers. Implement BEFORE and AFTER triggers on EMPLOYEE, COMPANY_INFO, and EMP_LOG tables using UPDATE operations.

BEFORE UPDATE Trigger

```
CREATE OR REPLACE TRIGGER trg_before_update
BEFORE UPDATE OF Salary ON EMPLOYEE
FOR EACH ROW
BEGIN
    :NEW.HRA := :NEW.Salary * 0.10;
END;
/
```

AFTER UPDATE Trigger

```
CREATE OR REPLACE TRIGGER trg_after_update
AFTER UPDATE OF Salary ON EMPLOYEE
FOR EACH ROW
BEGIN
    UPDATE COMPANY_INFO
    SET Total_Salary_Expenses =
    Total_Salary_Expenses - :OLD.Salary + :NEW.Salary;

    INSERT INTO EMP_LOG
    VALUES (
        :NEW.Emp_Id,
        :OLD.Salary,
        :NEW.Salary,
        SYSDATE,
        'UPDATED'
    );
END;
/
```